提取音频

```
ffmpeg.exe -i inputs/1723181542.mp4 -vn -acodec copy middle/1723181542/1723181542.aac
```

- `ffmpeg.exe`：运行FFmpeg程序。
- `-i inputs/1723181542.mp4`：指定输入文件为inputs目录下的1723181542.mp4文件。
- `-vn`：禁止视频流，只保留音频流进行处理。
- `-acodec copy`：使用原始编解码器（不重新编码）将音频流复制到输出文件中。
- `middle/1723181542/1723181542.aac`：指定输出文件路径和名称为middle目录下的1723181542子目录中的1723181542.aac

```
# 提取第1个音频流（索引从 0 开始，audio:0 表示第一个音频轨）
ffmpeg.exe -i .\goget.mp4 -vn -map 0:a:0  audio.wav

# 提取音频转为WAV格式
ffmpeg.exe -i .\goget.mp4 -vn  audio.wav
```

提取视频

```
ffmpeg.exe -i inputs/1723181542.mp4 -an -vcodec copy middle/1723181542/1723181542.mp4
```

- `ffmpeg.exe`：运行FFmpeg程序。
- `-i inputs/1723181542.mp4`：指定输入文件为inputs目录下的1723181542.mp4文件。
- `-an`：禁止音频流，只保留视频流进行处理。
- `-vcodec copy`：使用原始编解码器（不重新编码）将视频流复制到输出文件中。
- `middle/1723181542/1723181542.mp4`：指定输出文件路径和名称为middle目录下的1723181542子目录中的1723181542.mp4

截取音频

```
ffmpeg.exe -i middle/1723181542/1723181542.aac -ss 00:00:02.540 -t 6.000 refer/80.wav
```

使用了FFmpeg工具来从音频文件中提取一段指定时间范围的音频，并将其保存为另一个文件

- `ffmpeg.exe`：运行FFmpeg程序。
- `-i middle/1723181542/1723181542.aac`：指定输入文件为middle目录下的1723181542子目录中的1723181542.aac音频文件。
- `-ss 00:00:02.540`：指定起始时间为00时00分02秒540毫秒，表示从该时间点开始截取音频。
- `-t 6.000`：指定持续时间为6秒，表示截取6秒的音频。
- `refer/80.wav`：指定输出文件路径和名称为refer目录下的80.wav。

创建静音音频

```
ffmpeg.exe -f lavfi -i anullsrc=r=44100:cl=mono -t 25565ms
middle/1723181542/tmp/group_1_level_0_silence.wav
```

FFmpeg工具来生成一段指定时长的静音音频，并将其保存为一个文件

- `ffmpeg.exe`：运行FFmpeg程序。
- `-f lavfi -i anullsrc=r=44100:cl=mono`：指定输入源为lavfi（即FFmpeg内置的虚拟输入源），使用anullsrc过滤器生成一段无声音频。`r=44100`表示采样率为44.1kHz，`cl=mono`表示单声道（单声道）。
- `-t 25565ms`：指定持续时间为25565毫秒，即25.565秒，表示生成的静音音频的时长。
- `middle/1723181542/tmp/group_1_level_0_silence.wav`：指定输出文件路径和名称为middle目录下的1723181542子目录中的tmp目录下的group_1_level_0_silence.wav。

音频延迟播放

```
ffmpeg.exe -i middle/1723181542/audios/6.wav -af adelay=20300
middle/1723181542/tmp/adelay_6.wav
ffmpeg.exe -i middle/1723181542/audios/1.wav -af adelay=640
middle/1723181542/tmp/adelay_1.wav
ffmpeg.exe -i middle/1723181542/audios/2.wav -af adelay=2540
middle/1723181542/tmp/adelay_2.wav
ffmpeg.exe -i middle/1723181542/audios/5.wav -af adelay=15860
middle/1723181542/tmp/adelay_5.wav
ffmpeg.exe -i middle/1723181542/audios/4.wav -af adelay=11420
middle/1723181542/tmp/adelay_4.wav
ffmpeg.exe -i middle/1723181542/audios/3.wav -af adelay=6980
middle/1723181542/tmp/adelay_3.wav
```

使用FFmpeg工具对音频文件进行延迟处理，并将处理后的音频保存为一个新文件

- `ffmpeg.exe`：运行FFmpeg程序。
- `-i middle/1723181542/audios/6.wav`：指定输入文件路径和名称为middle目录下的1723181542子目录中的audios目录下的6.wav。
- `-af adelay=20300`：应用音频滤镜，其中`adelay`表示音频延迟效果，参数值20300表示延迟时间为20.3秒。
- `middle/1723181542/tmp/adelay_6.wav`：指定输出文件路径和名称为middle目录下的1723181542子目录中的tmp目录下的adelay_6.wav

```
# all=1 统一所有声道的延迟
ffmpeg.exe -i 2.wav -af adelay=10000:all=1 2_delay2.wav
```

音频第一次混流（分组合成音频）

```
ffmpeg -i middle/1723181542/tmp/group_1_level_0_silence.wav \
-i middle/1723181542/tmp/adelay_1.wav \
-i middle/1723181542/tmp/adelay_2.wav \
-i middle/1723181542/tmp/adelay_3.wav \
-i middle/1723181542/tmp/adelay_4.wav \
-i middle/1723181542/tmp/adelay_5.wav \
-i middle/1723181542/tmp/adelay_6.wav \
-filter_complex [0:a]volume=1[0];[1:a]volume=7[1];[2:a]volume=6[2];[3:a]volume=5[3];
[4:a]volume=4[4];[5:a]volume=3[5];[6:a]volume=2[6];[0][1][2][3][4][5]
[6]amix=inputs=7:duration=first[a] -map [a] \
-f wav middle/1723181542/tmp/group_1_level_0.wav
```

使用FFmpeg工具对多个音频文件进行处理，并将它们混合成一个输出音频文件

- `ffmpeg`：运行FFmpeg程序。
- `-i middle/1723181542/tmp/group_1_level_0_silence.wav`：指定输入文件路径和名称为middle目录下的1723181542子目录中的tmp目录下的group_1_level_0_silence.wav。
- `-i middle/1723181542/tmp/adelay_1.wav` 至 `-i middle/1723181542/tmp/adelay_6.wav`：分别指定其他六个输入文件的路径和名称。
- `-filter_complex [0:a]volume=1[0];[1:a]volume=7[1];[2:a]volume=6[2];[3:a]volume=5[3];[4:a]volume=4[4];[5:a]volume=3[5];[6:a]volume=2[6];[0][1][2][3][4][5][6]amix=inputs=7:duration=first[a] -map [a]`：应用音频滤镜，其中包括设置各个输入流的音量，然后使用amix滤镜将它们混合在一起。最后，通过-map选项选择要输出的音频流。
  - `[0:a]volume=1[0]`：对第一个输入流进行音量调整，将音量设置为原始大小，并将结果保存到标签 `[0]`。
  - `[1:a]volume=7[1]`：对第二个输入流进行音量调整，将音量增加到原来的 7 倍，并将结果保存到标签 `[1]`。
  - `[2:a]volume=6[2]`：对第三个输入流进行音量调整，将音量增加到原来的 6 倍，并将结果保存到标签 `[2]`。
  - `[3:a]volume=5[3]`：对第四个输入流进行音量调整，将音量增加到原来的 5 倍，并将结果保存到标签 `[3]`。
  - `[4:a]volume=4[4]`：对第五个输入流进行音量调整，将音量增加到原来的 4 倍，并将结果保存到标签 `[4]`。
  - `[5:a]volume=3[5]`：对第六个输入流进行音量调整，将音量增加到原来的 3 倍，并将结果保存到标签 `[5]`。
  - `[6:a]volume=2[6]`：对第七个输入流进行音量调整，将音量增加到原来的 2 倍，并将结果保存到标签 `[6]`。
  - `[0][1][2][3][4][5][6]amix=inputs=7:duration=first[a]`：将标签为 `[0]` 到 `[6]` 的音频流使用 amix 混合滤镜进行混合，指定输入数量为 7，且以第一个输入的时长作为输出时长，并将结果保存到标签 `[a]`。
  - `-map [a]`：选择标签为 `[a]` 的音频流作为输出

- `-f wav middle/1723181542/tmp/group_1_level_0.wav`：指定输出文件格式为wav，并将输出保存为middle目录下的1723181542子目录中的tmp目录下的group_1_level_0.wav。

创建静音视频（为合成组的结果创建静音音频）

```
ffmpeg.exe -f lavfi -i anullsrc=r=44100:cl=mono -t 25565ms
middle/1723181542/tmp/group_1_level_1_silence.wav
```

- `ffmpeg.exe`：FFmpeg 可执行文件的名称。
- `-f lavfi`：指定输入格式为 lavfi，即使用 libavfilter 库提供的过滤器作为输入。
- `-i anullsrc=r=44100:cl=mono`：指定输入源为一个空白音频流，使用 `anullsrc` 过滤器生成。该音频流具有 44100Hz 的采样率（r=44100）和单声道（cl=mono）。
- `-t 25565ms`：设置输出音频的持续时间为 25565 毫秒，即25.565秒。
- `middle/1723181542/tmp/group_1_level_1_silence.wav`：指定输出文件的路径和文件名

音频延迟播放（组音频）

```
ffmpeg.exe -i middle/1723181542/tmp/group_1_level_0.wav -af adelay=0
middle/1723181542/tmp/adelay_group_1_level_0.wav
```

使用 FFmpeg 工具对音频文件进行延迟处理的命令

- `ffmpeg.exe`：FFmpeg 可执行文件的名称。
- `-i middle/1723181542/tmp/group_1_level_0.wav`：指定输入文件为 `group_1_level_0.wav`，该文件位于路径 `middle/1723181542/tmp/` 下。
- `-af adelay=0`：指定音频滤镜，其中 `adelay=0` 表示不进行任何延迟处理，即原始音频保持不变。
- `middle/1723181542/tmp/adelay_group_1_level_0.wav`：指定输出文件的路径和文件名

音频第二次混流

```
ffmpeg.exe -i middle/1723181542/tmp/group_1_level_1_silence.wav -i
middle/1723181542/tmp/adelay_group_1_level_0.wav -filter_complex [0:a]volume=1[0];
[1:a]volume=2[1];[0][1]amix=inputs=2:duration=first[a] -map [a] -f mp3
middle/1723181542/tmp/group_1_level_1.mp3
```

使用 FFmpeg 工具对两个音频文件进行处理，最终生成一个混合音频文件

- `ffmpeg.exe`：FFmpeg 可执行文件的名称。
- `-i middle/1723181542/tmp/group_1_level_1_silence.wav`：指定第一个输入文件为 `group_1_level_1_silence.wav`，该文件位于路径 `middle/1723181542/tmp/` 下。
- `-i middle/1723181542/tmp/adelay_group_1_level_0.wav`：指定第二个输入文件为 `adelay_group_1_level_0.wav`，该文件也位于路径 `middle/1723181542/tmp/` 下。
- `-filter_complex [0:a]volume=1[0];[1:a]volume=2[1];[0][1]amix=inputs=2:duration=first[a] -`

`map [a] -f mp3 middle/1723181542/tmp/group_1_level_1.mp3`：指定复杂的滤镜图表达式。在这里，我们进行了以下操作：

- `[0:a]volume=1[0]` 表示对第一个输入音频应用音量调整滤镜，将音量设置为 1（保持不变），并将输出命名为 `[0]`。
- `[1:a]volume=2[1]` 表示对第二个输入音频应用音量调整滤镜，将音量设置为 2（增大一倍），并将输出命名为 `[1]`。
- `[0][1]amix=inputs=2:duration=first[a]` 表示对两个音频流进行混合，使用 `amix` 滤镜，指定了输入数量为 2，持续时间以第一个输入的时长为准，并将输出命名为 `[a]`。
- `-map [a]` 表示选择滤镜处理后的音频流作为最终输出。
- `-f mp3 middle/1723181542/tmp/group_1_level_1.mp3` 指定输出文件格式为 mp3，并将结果保存到指定路径下的 `group_1_level_1.mp3` 文件中

音视频混流

```
ffmpeg.exe -i middle/1723181542/1723181542.mp4 -i middle/1723181542/tmp/group_1_level_1.mp3
-c:v copy -c:a copy middle/1723181542/tmp/1723181542.mp4
```

使用 FFmpeg 工具对两个媒体文件进行处理，并生成一个新的媒体文件。

- `ffmpeg.exe`：FFmpeg 可执行文件的名称。
- `-i middle/1723181542/1723181542.mp4`：指定第一个输入文件为 `1723181542.mp4`，该文件位于路径 `middle/1723181542/` 下。
- `-i middle/1723181542/tmp/group_1_level_1.mp3`：指定第二个输入文件为 `group_1_level_1.mp3`，该文件位于路径 `middle/1723181542/tmp/` 下。
- `-c:v copy -c:a copy`：使用相同的视频编解码器和音频编解码器进行复制操作，即将输入流中的视频和音频数据直接复制到输出流中，不进行任何转码或修改。
- `middle/1723181542/tmp/1723181542.mp4`：指定输出文件名为 `1723181542.mp4`，保存在路径 `middle/1723181542/tmp/` 中。

视频加字幕

```
ffmpeg.exe -i middle/1723181542/tmp/1723181542.mp4 -vf subtitles=srts/1723181542_split.srt
outputs/1723181542.mp4
```

使用 FFmpeg 工具对一个视频文件进行处理，并在视频上添加字幕。

- `ffmpeg.exe`：FFmpeg 可执行文件的名称。
- `-i middle/1723181542/tmp/1723181542.mp4`：指定输入文件为 `1723181542.mp4`，该文件位于路径 `middle/1723181542/tmp/` 下。
- `-vf subtitles=srts/1723181542_split.srt`：通过 `-vf` 参数指定视频滤镜，其中 `subtitles` 表示要添加字幕，后面的 `srts/1723181542_split.srt` 指定了字幕文件的路径和名称。这意味着要将位于路径 `srts/1723181542_split.srt` 下的字幕添加到视频中。
- `outputs/1723181542.mp4`：指定输出文件名为 `1723181542.mp4`，保存在路径 `outputs/` 中。

补充

更改视频播放速度

将视频的播放速度减慢为原来的 1/2 倍（即慢放 2 倍）

```
ffmpeg -i input.mp4 -filter:v "setpts=2*PTS" -filter:a "atempo=0.5" output.mp4
```

各参数详细解读

1. 基础参数：`ffmpeg` & `-i input.mp4`

- `ffmpeg`：调用 FFmpeg 主程序；
- `-i input.mp4`：指定输入文件为 `input.mp4`（`-i` 是 input 的缩写，是 FFmpeg 指定输入源的核心参数）。

2. 视频滤镜：`-filter:v "setpts=2*PTS"`

- `-filter:v`：简写为 `-vf`，表示对视频流（video） 应用滤镜；
- `setpts`：是 FFmpeg 中用于调整视频帧时间戳（PTS） 的滤镜（PTS = Presentation Time Stamp，显示时间戳，决定帧何时显示）；
- `2*PTS`：将每帧的显示时间戳设置为原来的 2 倍。

3. 音频滤镜：`-filter:a "atempo=0.5"`

- `-filter:a`：简写为 `-af`，表示对音频流（audio） 应用滤镜；
- `atempo`：是 FFmpeg 中用于调整音频播放速度 / 音调的滤镜（核心特性：调整速度时不改变音频的音调）；
- `0.5`：音频的播放速度系数，设置为 0.5 表示音频速度减慢为原来的 1/2 倍。

4. `atempo` 滤镜的限制与解决方案

`atempo` 有一个参数范围限制：速度系数必须在 `0.5` 到 `2.0` 之间（即只能将音频速度调整为原来的 0.5~2 倍）。如果需要超出这个范围（如慢放 4 倍，即 `atempo=0.25`），可以通过多次叠加 `atempo` 实现：

```
# 音频慢放 4 倍（0.5 × 0.5），视频对应 setpts=4*PTS
ffmpeg -i input.mp4 -filter:v "setpts=4*PTS" -filter:a "atempo=0.5,atempo=0.5" output.mp4
```

获取音频长度

从文件层面获取时长，比如：一个文件包含多个流（音频和视频）

```
# 获取WAV音频时长，ffprobe 读取元数据，仅限WAV等 无压缩的 PCM 格式
ffprobe.exe -v error -show_entries format=duration -of default=noprint_wrappers=1:nokey=1
audio.wav
```

各参数详细解读

1. `ffprobe.exe`

- 是 FFmpeg 配套的媒体元数据分析工具（而非 ffplay/ffmpeg），专门用于解析音频 / 视频文件的格式、流信息、编码参数等元数据，不进行转码或播放操作。
- Windows 系统下为 `ffprobe.exe`，Linux/Mac 系统下直接为 `ffprobe`。

2. `-v error`

- `-v` 是 `loglevel` 的缩写，用于控制日志输出的详细程度。
- `error` 表示仅输出错误信息，屏蔽所有无关的日志（如版本信息、调试信息、警告信息），让输出结果只保留需要的时长数据，避免冗余。

3. `-show_entries format=duration`

- 这是核心数据提取参数，用于指定要获取的元数据字段：
  - `show_entries`：表示 “显示指定的条目 / 字段”；
  - `format=duration`：限定提取格式层（format） 的 `duration` 字段（即文件的总时长）。
  - 补充：ffprobe 将媒体文件的元数据分为 `format`（整体格式信息）和 `stream`（流信息，如音频流、视频流）两层，这里直接取 `format` 层的时长，对 WAV 文件而言精准度高。

4. `-of default=noprint_wrappers=1:nokey=1`

- `-of` 是 `output_format` 的缩写，用于指定输出格式，控制返回数据的呈现形式：
  - `default`：使用 ffprobe 的默认输出格式（键值对格式）；
  - `noprint_wrappers=1`：禁用默认的 “包装器” 输出（如避免出现 `[FORMAT]`、`[/FORMAT]` 这类包裹性标签）；
  - `nokey=1`：不输出字段的键名（即只输出 `duration` 的值，不输出 `duration=` 前缀）。
  - 组合效果：让输出结果仅保留时长的数值，去除所有格式标签和键名，达到极简的纯数值输出。

5. `audio.wav`

- 指定要分析的目标文件路径（这里是 WAV 音频文件），可以是相对路径或绝对路径（如 `D:/music/test.wav`）

获取流时长

从流的层面获取时长，可以指定获取某个流的时长，但是可能和文件层面获取到的时长有细微的差别

提取 WAV 音频文件中`音频流`的持续时长（以秒为单位，含小数精度），是针对流级别（stream） 元数据的精准查询方式

```
# 获取音频流时长
ffprobe.exe -v error -show_entries stream=duration -of default=noprint_wrappers=1:nokey=1
audio.wav
```

各参数详细解读

1. `ffprobe.exe`

FFmpeg 配套的媒体元数据分析工具，专注于解析音频 / 视频文件的流信息、编码参数、时长等元数据，不涉及转码或播放操作（Windows 下为 `.exe` 后缀，Linux/Mac 下直接为 `ffprobe`）。

2. `-v error`

- `-v`（`loglevel` 的缩写）：控制日志输出的详细程度；
- `error`：仅输出错误信息，屏蔽版本信息、调试日志、警告等冗余内容，让输出结果只保留核心的时长数值。

3. `-show_entries stream=duration`

这是核心数据提取参数，也是与之前 `format=duration` 命令的关键区别：

- `show_entries`：指定要提取的元数据字段；
- `stream=duration`：限定提取流级别（stream） 的 `duration` 字段（即单个媒体流的时长）。对 WAV 文件而言，只有一个音频流，因此这里提取的就是音频流的时长；若为视频文件（如 MP4），则会返回所有流（视频流、音频流）的时长（可通过 `-select_streams` 限定流类型）。

4. `-of default=noprint_wrappers=1:nokey=1`

- `-of`（`output_format` 的缩写）：指定输出格式；
- `default`：使用 ffprobe 的默认输出格式（键值对格式）；
- `noprint_wrappers=1`：禁用 `[STREAM]` / `[/STREAM]` 这类包裹性标签；
- `nokey=1`：不输出字段的键名（即只输出时长数值，不输出 `duration=` 前缀）。组合效果：输出纯数值的时长，无任何格式冗余，便于脚本 / 代码直接解析。

5. `audio.wav`

指定要分析的目标 WAV 文件路径（相对路径 / 绝对路径均可）

6. 其他参数

`-select_streams a:0`：`a` 代表音频流（audio），`0` 代表第 0 个音频流（通常只有一个）

音频拼接

多个 WAV 文件的参数（采样率、声道数等）不一致，可通过音频滤镜自动统一参数后拼接：

```
ffmpeg.exe -i 2.wav -i 3.wav -i 1.wav -filter_complex "[0:a][1:a]
[2:a]concat=n=3:v=0:a=1[outa]" -map "[outa]" 4.wav
```

- **参数说明**：
  - `filter_complex`：启用复杂滤镜链；
  - `[0:a][1:a][2:a]`：分别引用第 1、2、3 个输入文件的音频流；
  - `concat=n=3:v=0:a=1`：`n=3` 表示拼接 3 个音频流，`v=0` 禁用视频，`a=1` 启用音频；

- `[outa]`：为拼接后的音频流命名，后续通过 `-map` 映射到输出。

音频混合（叠加，同时播放）

```
ffmpeg.exe -i 2.wav -i 3.wav -i 1.wav -filter_complex "[0:a][1:a]
[2:a]amix=inputs=3:duration=longest[outa]" -map["outa"] 4.wav
```

参数说明：

- `amix=inputs=3`：指定混合 3 个音频输入；
- `duration=longest`：混合后的音频时长取最长的输入音频（可选值：`shortest`/`longest`/`first`）